

RadioChat – APRS Chat aplikacija putem KISS TCP

Leon Baždar

GitHub repozitorij: <https://github.com/mrleongalaxyum/RadioChat>

1. Opis aplikacije

RadioChat je desktop aplikacija koja omogućuje dvosmjernu tekstualnu komunikaciju putem APRS-a koristeći DireWolf softverski TNC (<https://github.com/wb2osz/direwolf/blob/master/doc/README.md>) i KISS TCP protokol. Sučelje aplikacije je jednostavno i nalik klasičnoj chat aplikaciji. Korisnik može slati poruke na odabranu APRS destinaciju, pratiti dolazne poruke u stvarnom vremenu te ih spremati u lokalnu bazu podataka.

2. Tehnologije korištene u implementaciji

Ova aplikacija objedinjuje više tehnologija:

Tehnologija / Koncept	Primjena u aplikaciji
TCP/IP komunikacija	Korištenje Python socket modula za vezu s DireWolf KISS TCP poslužiteljem na portu 8001.
AX.25 i APRS format	Enkodiranje i dekodiranje poruka u skladu s AX.25 protokolom, korištenjem APRS poruka :DEST:MSG. (https://ax25.net/)
KISS protokol	Slanje i primanje frameova pomoću KISS enkapsulacije (FEND, FESC). (https://ax25.net/kiss.aspx)
GUI (Tkinter)	Izrada jednostavnog chat sučelja s unosom, prikazom i kontrolama.
SQLite baza podataka	Pohrana svih primljenih i poslanih poruka za arhiviranje.
Višedretvenost (threading)	Primanje poruka u pozadini kako bi GUI ostao responzivan.
Regex validacija	Validacija korisničkog unosa (pozivni znakovi, SSID, destinacija).

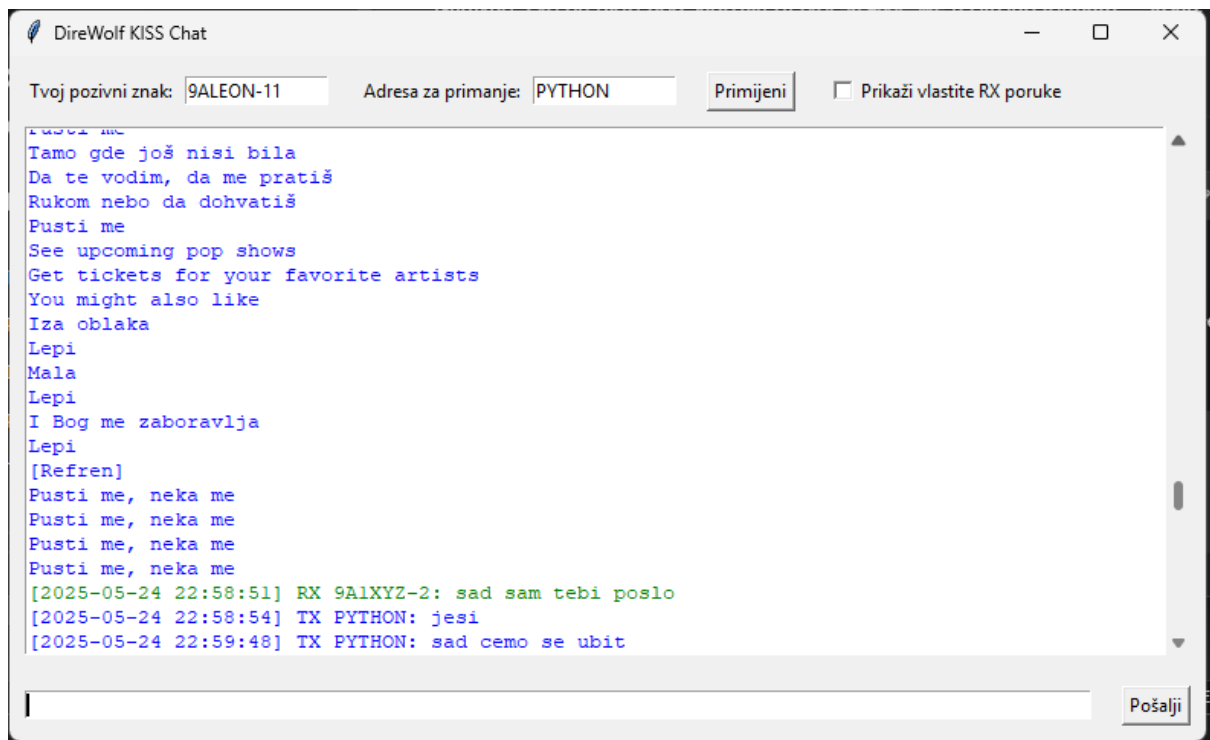
3. Arhitektura i način rada

1. Korisnik pokreće DireWolf.exe s ispravno konfiguriranim ulaznim i uredajima u direwolf.conf (<https://github.com/wb2osz/direwolf-doc>)
2. Korisnik pokreće aplikaciju (RadioChat.exe) koja otvara TCP vezu na 127.0.0.1:8001, gdje DireWolf izlaže KISS TCP sučelje.
3. Poruke se enkodiraju u AX.25 UI-frame format i omotaju KISS protokolom.
4. GUI omogućava unos pozivnog znaka, destinacije te poruke. Poruke se šalju pritiskom na gumb ili tipku Enter.
5. Primanje poruka odvija se u pozadinskom threadu. Poruke koje odgovaraju zadanom pozivnom znaku (destinaciji) prikazuju se u GUI prozoru.
6. Svaka poruka (RX i TX) pohranjuje se u SQLite bazu podataka messages.db zajedno s vremenom, adresom i smjerom.

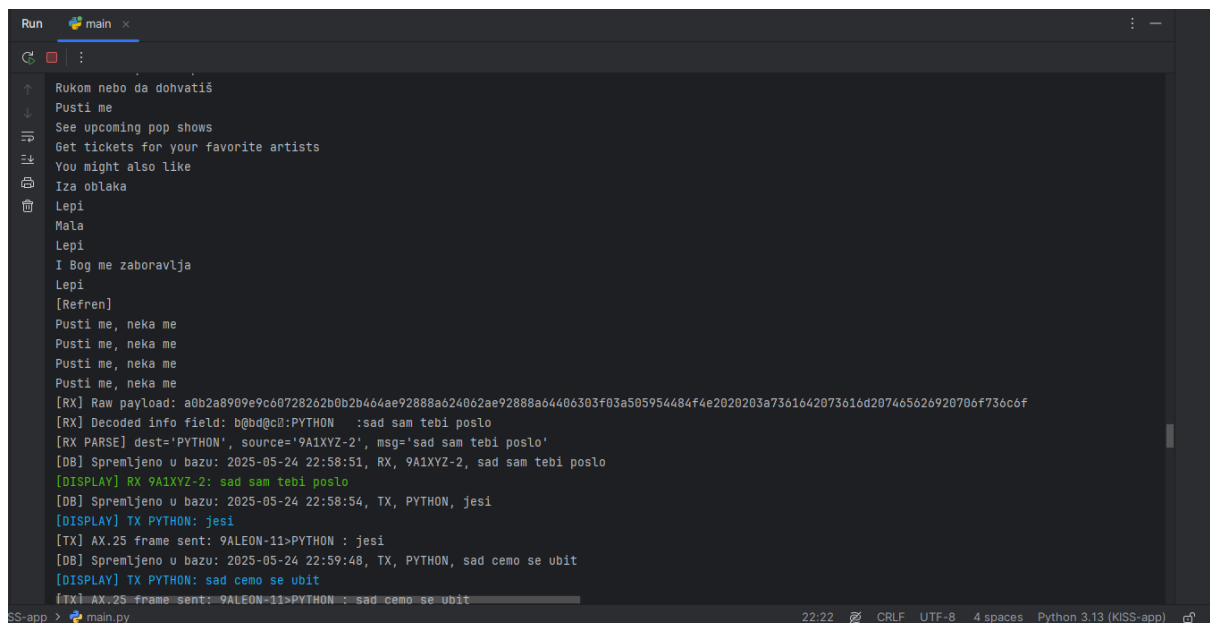
4. Sučelje

Sučelje aplikacije (GUI)

- Gornji red omogućava postavljanje pozivnog znaka i destinacije
- Glavni prozor prikazuje poruke s vremenskom oznakom i bojama (TX – plavo, RX – zeleno)
- Donji dio omogućuje unos i slanje poruka



Terminal Sučelje



5. Povezivanje sustava na računalo:

Obzirom da koristimo AFSK1200 modulaciju, potrebno je povezati jedan audio izlaz s računala na Baofeng UV5R. Zatim je potrebno upalite SDR# softver te DireWolf server te konfigurirati audio izlaz sa SDR# na input kanal DireWolfa te output kanal DireWolfa konfigurirati na izlaz zvučne kartice. Obzirom da je usmjeravanje zvučnih tokova na Windows operativnom sustavu kompleksno, korišten je audio softver Voicemeeter Potato kako bi se postigla gore navedena konfiguracija.

6. Istaknuti dijelovi programskog koda:

Enkodiranje AX.25 poruke

Poruke, prije modulacije AFSK1200 postupkom i emitiranja na audio ulaz Baofeng UV5R ručne radio stanice budu enkodirane u format AX.25 poruka. <https://ax25.net/>

```
def encode_ax25_frame(src, dest, message):

    def ax25_address(callsign, ssid, last=False):
        callsign = callsign.ljust(6)
        addr = bytes([(ord(c) << 1) for c in callsign])
        ssid_byte = 0b01100000 | ((int(ssid) & 0x0F) << 1)
        if last:
            ssid_byte |= 1
        return addr + bytes([ssid_byte])

    src_name, src_ssid = (src.split('-') + ['0'][:2])
    dest_name, dest_ssid = (dest.split('-') + ['0'][:2])
    frame = b''
    frame += ax25_address(dest_name, dest_ssid)
    frame += ax25_address(src_name, src_ssid)
    frame += ax25_address('WIDE1', 1)
    frame += ax25_address('WIDE2', 1, last=True)
    frame += b'\x03'
    frame += b'\xf0'
    frame += message.encode()
    return frame
```

KISS TCP poruke

<https://ax25.net/kiss.aspx>

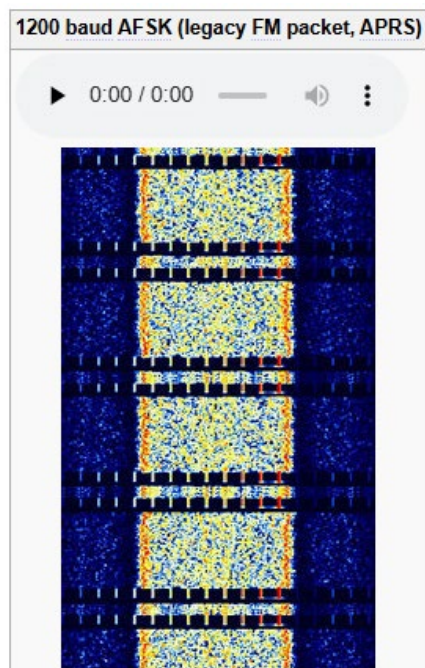
KISS (*eng. Keep It Simple Stupid*) protokol spaja se na localhost:8001 TCP portove za TX i RX poruka sa DireWolf poslužitelja. Koriste se funkcije `kiss_wrap(data)` i `kiss_unwrap(data)`

```
1. def kiss_unwrap(data):
    if data.startswith(b'\xC0') and data.endswith(b'\xC0'):
        return data[1:-1]
    return data

def kiss_wrap(data):
    return b'\xC0\x00' + data + b'\xC0'
```

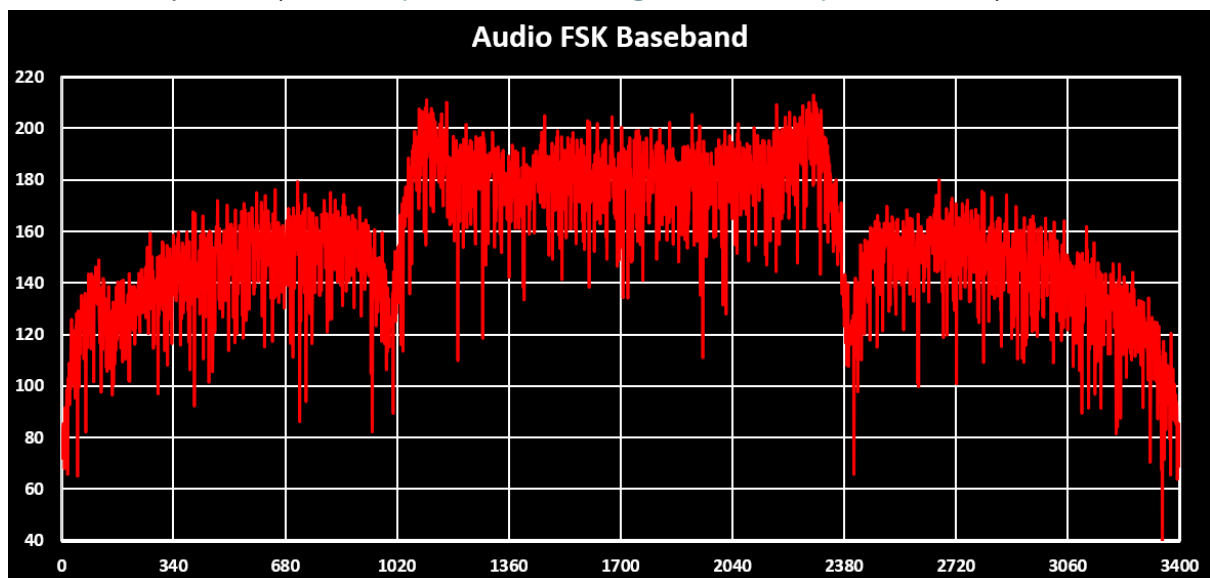
AFSK 1200

<https://www.sigidwiki.com/wiki/PACKET>



AFSK1200 (eng. *Audio Frequency Shift Keying*) je protokol koji enkodira podatke pomoću dvije frekvencije. Frekvencija 1200Hz predstavlja logičku jedinicu, 2200Hz nulu. Bitrate je u ovom slučaju 1200bps.

AFSK1200 spektar: (izvor: <https://notblackmagic.com/bitsnpieces/afsk/>)



Baza podataka

Baza podataka, ukoliko je nepostojeća, stvara se pod imenom messages.db i popunjava timestampom, smjerom komunikacije, adresom te sadržajem poruke, prema niže navedenom programskom kodu:

```
1. 1. def init_db():
    conn = sqlite3.connect(DB_NAME)
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS messages (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT NOT NULL,
            direction TEXT NOT NULL,
            address TEXT NOT NULL,
            message TEXT NOT NULL
        )
    ''')
    conn.commit()
    conn.close()

def save_message(timestamp, direction, address, message):
    try:
        conn = sqlite3.connect(DB_NAME)
        c = conn.cursor()
        c.execute('''
            INSERT INTO messages (timestamp, direction, address, message)
            VALUES (?, ?, ?, ?)
        ''', (timestamp, direction, address, message))
        conn.commit()
        conn.close()
        print(f"[DB] Spremljeno u bazu: {timestamp}, {direction}, {address}, {message}")
    except Exception as e:
        print(f"[DB ERROR] {e}")
```

7. Zaključak

Aplikacija RadioChat demonstrira kako se standardni Python alati mogu upotrijebiti za razvoj stvarnog komunikacijskog sustava. Primjenjuju se svi važni komunikacijski koncepti – od protokola niže razine (KISS, AX.25), do izrade korisničkog sučelja i lokalne obrade podataka. Aplikacija je modularna i lako proširiva, a može poslužiti i kao edukativni alat za rad s APRS protokolom.

8. Bonus sadržaj

Prethodni(neuspješni) pokušaji demodulacije snimljenih primljenih signala pomoću programskog paketa afskmodem.py

(<https://github.com/lavajuno/afskmodem/tree/main>)

Primljeni signal je sadržavao previse šuma pa ga modem nije mogao pravilno dekodirati. Modem je funkcionirao samo s audio datotekama generiranim istoimenim paketom, bez odašiljanja kroz komunikacijski kanal. Zato je on zamijenjen DireWolf programskim paketom koji to sve ima riješeno i omogućava robusnu (de)modulaciju

